

Sentencias para la manipulación de datos y transaccionalidad

Módulo: Fundamentos de bases de datos relacionales.

|AE3: Utilizar lenguaje de manipulación de datos DML para la modificación de los datos existentes en una base de datos dando solución a un problema planteado

Introducción

A medida que las aplicaciones digitales se vuelven más complejas, es crucial comprender cómo insertar, actualizar y eliminar datos de manera segura y eficiente, asegurando que los cambios realizados sean consistentes y estén protegidos ante posibles errores o fallos del sistema. En este contexto, **la manipulación de datos y la transaccionalidad** son aspectos fundamentales para garantizar la integridad y coherencia de la información.

En este manual, profundizaremos en las **operaciones de manipulación de datos** y exploraremos cómo gestionar transacciones para mantener la integridad referencial en nuestras bases de datos. Aprenderás a utilizar sentencias avanzadas de SQL para modificar datos mientras aplicas principios de transaccionalidad que garantizan la confiabilidad y estabilidad de las operaciones. Este conocimiento no solo optimiza el rendimiento de las aplicaciones, sino que también fortalece las bases para el desarrollo de **productos digitales robustos** y escalables.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Conocer los componentes principales de Data Manipulation Language (DML)
- Actualizar la información de una tabla
- Borrar información de una tabla
- Ingresar información a una tabla
- Utilizar de secuencias para asignar identificadores
- Insertar, actualizar y borrar datos con integridad referencial
- Conocer qué es una transacción y su importancia
- Conocer los principios ACID

Sentencias para la manipulación de datos y transaccionalidad

En el ámbito de las bases de datos relacionales, el Lenguaje de Manipulación de Datos (DML) es esencial para interactuar y gestionar la información almacenada. Con DML, es posible realizar operaciones fundamentales como consultar, insertar, actualizar y eliminar datos, lo que permite mantener la base de datos en un estado consistente y actualizado. Sin embargo, para utilizar estas sentencias de manera efectiva, es importante comprender algunos conceptos técnicos clave que garantizan la integridad y eficiencia de las operaciones.

Identificadores en Bases de Datos

Los identificadores son nombres únicos que se asignan a los elementos de una base de datos para distinguirlos de otros objetos. Se utilizan para identificar tablas, columnas, vistas, índices y otros componentes. En el contexto de DML, los identificadores permiten acceder y manipular datos de manera precisa. Es crucial seguir convenciones de nomenclatura claras y consistentes para evitar conflictos o ambigüedades, especialmente en bases de datos complejas.

Secuencias y asignación de identificadores únicos

Las secuencias son objetos de base de datos que generan números únicos de manera automática, facilitando la asignación de identificadores primarios en las tablas. Se utilizan comúnmente para crear claves primarias sin riesgo de duplicidad. Las funciones NEXTVAL y CURRVAL permiten trabajar con estos valores:

- **NEXTVAL**: Obtiene el siguiente número en la secuencia e incrementa su valor automáticamente.
- **CURRVAL**: Recupera el último valor generado sin incrementarlo, útil para obtener el identificador asignado en una inserción reciente.

Además, el uso de secuencias mejora el rendimiento al evitar bloqueos de concurrencia en entornos multiusuario y garantiza la integridad de los datos al generar identificadores consistentes.

Integridad Referencial y Relaciones entre Tablas

La integridad referencial asegura que las relaciones entre tablas permanezcan coherentes, evitando errores como registros huérfanos (datos sin relación válida). Se implementa mediante claves foráneas (FOREIGN KEY), que vinculan una columna de una tabla con la clave primaria de otra tabla. Este vínculo establece dependencias lógicas y mantiene la coherencia de los datos relacionados.

Para gestionar cambios en cascada, se pueden utilizar cláusulas como:

- **ON DELETE CASCADE**: Elimina automáticamente los registros relacionados cuando se borra una fila principal.
- **ON UPDATE CASCADE**: Actualiza los valores dependientes cuando se modifica la clave primaria.
- **SET NULL** y **SET DEFAULT**: Asignan un valor nulo o por defecto a las columnas relacionadas al modificar o eliminar registros principales.
- **RESTRICT**: Evita la eliminación o modificación si existen datos dependientes, protegiendo la integridad referencial.

Estas opciones permiten automatizar la gestión de dependencias, reduciendo el riesgo de errores y mejorando el mantenimiento de datos consistentes.

Aprovechando el potencial de DML

Dominar las sentencias de DML y comprender el funcionamiento de identificadores, secuencias e integridad referencial no solo mejora la eficiencia al trabajar con datos, sino que también garantiza la confiabilidad y precisión de las operaciones en las bases de datos. Al aplicar estas prácticas, se pueden desarrollar productos digitales robustos capaces de manejar grandes volúmenes de información de manera segura y consistente.

Data Manipulation Language (DML)

El lenguaje de manipulación de datos (DML) es un **sublenguaje del lenguaje de consulta estructurado (SQL)** que **se utiliza para manipular y trabajar con los datos almacenados en una base de datos**. DML proporciona comandos y

sintaxis que permiten a los usuarios consultar, insertar, actualizar y eliminar datos dentro de tablas de bases de datos.

Se centra principalmente en la modificación de los datos de la base de datos más que en la estructura de la propia base de datos.

Las principales operaciones que proporciona DML son:

- **SELECT:** La sentencia SELECT recupera datos de una o varias tablas de la base de datos. Permite especificar las columnas y filas que se desean recuperar en función de determinadas condiciones mediante la cláusula
- **WHERE.** Es el comando más utilizado para consultar datos.
- **INSERT:** La sentencia INSERT se utiliza para añadir nuevas filas de datos a una tabla. Permite especificar los valores de cada columna o seleccionarlos de otra tabla.
- **UPDATE:** La sentencia UPDATE se utiliza para modificar los datos existentes en una tabla. Permite actualizar columnas específicas en una o más filas basándose en condiciones especificadas.
- **DELETE:** La sentencia DELETE se utiliza para eliminar una o varias filas de una tabla. Permite especificar condiciones para determinar qué filas eliminar.
- Los comandos DML suelen ejecutarse mediante sentencias SQL a través de sistemas de gestión de bases de datos (SGBD) como Oracle, MySQL, SQL Server o PostgreSQL. Estos comandos proporcionan la funcionalidad necesaria para interactuar con los datos almacenados en la base de datos y manipularlos.

Actualizando la información de una tabla

Para actualizar la información de una tabla mediante el lenguaje de manipulación de datos (DML) de SQL, se puede utilizar la sentencia **UPDATE**.

Esta es la sintaxis básica

```
UPDATE table_name
SET column1 = value1, column2 =
value2, ... WHERE condition;
```

Vamos a desglosar los distintos componentes:

- **nombre_tabla:** El nombre de la tabla que desea actualizar.
- **columna1, columna2, ...:** Las columnas que desea actualizar.
- **valor1, valor2, ...:** Los nuevos valores que desea establecer para las columnas correspondientes.
- **Condición WHERE:** Una condición opcional que especifica qué filas deben actualizarse. Si omite la cláusula WHERE, se actualizarán todas las filas de la tabla.

A continuación se muestra un ejemplo para ilustrar cómo utilizar la sentencia UPDATE:

Supongamos que tenemos una tabla llamada empleados con las columnas employee_id, first_name, last_name y salary. Queremos actualizar el salario del empleado con employee_id igual a 123 a 5000.

```
UPDATE
employees    SET
salary = 5000
WHERE employee_id = 123;
```

Esta consulta actualizará la columna de salario del empleado con employee_id 123 a 5000.

Borrando información de una tabla

Para actualizar la información de una tabla mediante el lenguaje de manipulación de datos (DML) de SQL, puede utilizar la sentencia **DELETE**. Esta es la sintaxis básica

```
DELETE          FROM
table_name      WHERE
condition;
```

Desglosemos los distintos componentes:

- **table_name:** El nombre de la tabla de la que desea eliminar datos.

- **Condición WHERE:** Una condición opcional que especifica qué filas deben eliminarse. Si omite la cláusula WHERE, se eliminarán todas las filas de la tabla.

A continuación se muestra un ejemplo para ilustrar cómo utilizar la sentencia DELETE:

Supongamos que tenemos una tabla llamada **empleados** con las columnas **employee_id**, **first_name**, **last_name** y **salary**. Queremos borrar el empleado con employee_id igual a 123.

```
DELETE FROM employees
WHERE employee_id =
123;
```

Esta consulta eliminará la fila de la tabla de empleados cuyo employee_id sea 123.

Es importante tener en cuenta que la sentencia DELETE elimina toda la fila de la tabla. Si sólo desea eliminar los valores de columnas específicas pero conservar la fila, puede utilizar la sentencia UPDATE y establecer los valores en NULL o en un valor predeterminado adecuado.

Ingresando información a una tabla

Para actualizar la información de una tabla mediante el lenguaje de manipulación de datos (DML) de SQL, puede utilizar la sentencia **INSERT INTO**.

Esta es la sintaxis básica

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Veamos qué significan cada uno de estos valores:

- ⇒ **table_name:** El nombre de la tabla en la que desea insertar datos.
- ⇒ **(columna1, columna2, ...):** Nombres de columna opcionales, si desea especificar las columnas explícitamente. Si omite esta parte, deberá proporcionar valores para todas las columnas de la tabla, y el orden de los valores deberá coincidir con el orden de las columnas de la tabla.

- **VALORES (value1, value2, ...)**: Los valores que desea insertar en las respectivas columnas. El orden de los valores debe coincidir con el orden de columnas especificado o con el orden de columnas de la tabla si no especificó explícitamente las columnas.

Veamos un ejemplo para ilustrar cómo utilizar la sentencia INSERT INTO:

Supongamos que tenemos una tabla llamada empleados con las columnas employee_id, first_name, last_name y salary. Queremos insertar una nueva fila con los siguientes valores: 456 para employee_id, 'John' para first_name, 'Doe' para last_name, y 5000 para salary

```
INSERT INTO employees (employee_id, first_name,
last_name, salary) VALUES (456, 'John', 'Doe', 5000);
```

Esta consulta insertará una nueva fila en la tabla de empleados con los valores especificados.

Hay que tener en cuenta que si va a insertar varias filas a la vez, puede utilizar una lista de conjuntos de valores separados por comas en la cláusula VALUES. Por ejemplo:

```
INSERT INTO employees (employee_id, first_name,
last_name, salary) VALUES (456, 'John', 'Doe', 5000),
(789, 'Jane', 'Smith', 6000),
(123, 'Bob', 'Johnson', 5500);
```

Esta consulta insertará tres filas en la tabla de empleados con los valores especificados.

Identificadores de bases de datos en SQL

En SQL, los **identificadores** se utilizan para representar diversos objetos de la **base de datos**, como tablas, columnas, vistas, índices y otros. Un **identificador** es simplemente **un nombre dado a estos objetos para identificarlos y referenciarlos de forma única** dentro de la base de datos.

Estos son algunos puntos importantes que hay que saber sobre los identificadores SQL:

- **Caracteres válidos:** Los identificadores suelen estar formados por caracteres alfanuméricos (A-Z, a-z, 0-9) y guiones bajos (_). El primer carácter de un identificador debe ser una letra o un guión bajo.
- **Distinción entre mayúsculas y minúsculas:** Los identificadores SQL pueden distinguir entre mayúsculas y minúsculas o no, dependiendo del sistema de base de datos y su configuración. Por ejemplo, en algunas bases de datos, los identificadores no distinguen entre mayúsculas y minúsculas por defecto, mientras que en otras sí. Es importante consultar la documentación o la configuración de su sistema de base de datos específico.
- **Palabras clave reservadas:** SQL tiene un conjunto de palabras clave reservadas que tienen significados especiales dentro del lenguaje. Estas palabras clave no pueden utilizarse como identificadores a menos que vayan entre comillas dobles (") o comillas inversas (') en algunos sistemas de bases de datos.
- **Límite de longitud:** Suele haber un límite máximo de longitud para los identificadores en SQL, que varía en función del sistema de base de datos. Se recomienda que los identificadores sean concisos y significativos y que se respeten las restricciones específicas de longitud.
- **Buenas prácticas:** Es una buena práctica elegir nombres descriptivos y significativos para los identificadores con el fin de mejorar la legibilidad y el mantenimiento del código. Evite utilizar caracteres especiales, espacios o palabras clave reservadas como identificadores para evitar conflictos y mejorar la portabilidad entre distintos sistemas de bases de datos.

A continuación se muestran algunos ejemplos de identificadores SQL:

- **Nombre de tabla:** clientes, pedidos, productos.
- **Nombre de columna:** first_name, last_name, email_address.
- **Nombre del índice:** idx_orders_customer_id, ix_products_name.

En general, los identificadores SQL desempeñan un papel crucial a la hora de identificar y referenciar objetos de base de datos dentro de sentencias SQL, lo que permite realizar operaciones eficaces de manipulación y recuperación de datos.

Utilización de secuencias para asignar identificadores

En SQL, las secuencias **se utilizan** a menudo **para generar automáticamente identificadores únicos para las columnas**. Una secuencia **es un objeto que genera una serie de valores numéricos basados en un incremento y un valor inicial especificados**. Se pueden utilizar secuencias para asignar identificadores únicos a columnas, como claves primarias, al insertar nuevos registros en una tabla.

La sintaxis y funcionalidad específicas de las secuencias pueden variar ligeramente entre los distintos sistemas de bases de datos. A continuación se ofrece una visión general de cómo utilizar secuencias en SQL:

Crear una secuencia: En primer lugar, debe crear una secuencia. La sintaxis para crear una secuencia puede variar dependiendo del sistema de base de datos que esté utilizando. Aquí hay un ejemplo usando PostgreSQL:

```
CREATE SEQUENCE sequence_name
  START WITH start_value
  INCREMENT BY increment_value
  MINVALUE min_value

  MAXVALUE
  max_value CYCLE
  | NO CYCLE;
```

- **sequence_name:** El nombre de la secuencia.
- **valor_inicial:** El valor inicial de la secuencia.
- **valor_incremento:** El incremento entre cada valor generado por la secuencia.
- **valor_mínimo (opcional):** El valor mínimo que puede generar la secuencia.
- **valor_máx (opcional):** El valor máximo que puede generar la secuencia.
- **CICLO | NO CICLO (opcional):** Determina si la secuencia debe volver a empezar desde el principio (CICLO) o dejar de generar valores (NO CICLO) al alcanzar el valor máximo o mínimo.

Utilizar la secuencia en una sentencia INSERT: Después de crear la secuencia, puede utilizarla en una sentencia INSERT para asignar automáticamente valores a una columna. He aquí un ejemplo:

```
INSERT INTO table_name (column_name1, column_name2,
...) VALUES (nextval('sequence_name'), value2,
...);
```

- **nombre_tabla:** El nombre de la tabla en la que se está insertando.
- **nombre_columna1:** El nombre de la columna a la que quieres asignar el valor de la secuencia.
- **nextval('nombre_secuencia'):** La función nextval recupera el siguiente valor de la secuencia especificada.

Al utilizar la secuencia en la sentencia INSERT, la columna asignada a la secuencia se rellenará automáticamente con un valor único de la secuencia.

Las secuencias proporcionan una forma eficaz y fiable de generar identificadores únicos en SQL, garantizando la integridad de los datos y evitando conflictos cuando se insertan varios registros simultáneamente.

Insertar, actualizar y borrar datos con integridad referencial ↓

Integridad referencial

En SQL es un concepto que **garantiza la coherencia e integridad de las relaciones entre tablas** mediante la aplicación de determinadas reglas y restricciones. Mantiene la exactitud y validez de los datos preservando la integridad de las relaciones entre tablas, normalmente mediante el uso de restricciones de clave externa.

La integridad referencial garantiza que las relaciones entre tablas se mantengan de las siguientes maneras:

- **Clave primaria - Relación de clave foránea:** La integridad referencial garantiza que los valores de una columna de clave foránea de una tabla se correspondan con los valores de la columna de clave primaria de otra tabla. Esto significa que cada valor de clave foránea debe tener un valor de clave primaria coincidente en la tabla referenciada.
- **Coherencia de las relaciones:** La integridad referencial evita los registros huérfanos al garantizar que si se elimina una fila de la tabla padre (tabla referenciada), todas las filas relacionadas en la tabla hija (tabla referenciada) también se eliminan o modifican en consecuencia en función

de las acciones en cascada especificadas.

- **Precisión y validez de los datos:** La integridad referencial ayuda a mantener la exactitud y validez de los datos evitando la inserción de valores no válidos o inexistentes en columnas de clave externa. Garantiza la coherencia de los datos en todas las tablas relacionadas.

Al aplicar la integridad referencial, las bases de datos SQL garantizan que las relaciones entre las tablas sean coherentes y que las operaciones de datos mantengan la integridad de estas relaciones.

Restricciones en una tabla

Las restricciones (constraints) en una tabla SQL son reglas y condiciones que se aplican a las columnas o combinaciones de columnas para garantizar la integridad y consistencia de los datos. Estas restricciones definen reglas que deben cumplirse para insertar, actualizar o eliminar datos en una tabla.

Las restricciones más comunes en una tabla SQL son:

- **Primary Key Constraint (Restricción de clave primaria):** Esta restricción garantiza que una columna o combinación de columnas sirva como clave primaria, lo que implica que los valores en esa columna o combinación de columnas deben ser únicos e irrepetibles en la tabla.
- **Foreign Key Constraint (Restricción de clave externa):** Esta restricción establece una relación entre una columna o combinación de columnas de una tabla (clave externa) y la clave primaria de otra tabla. Garantiza que los valores en la columna o combinación de columnas de la clave externa existan en la tabla referenciada como clave primaria o como un valor único.
- **Unique Constraint (Restricción de unicidad):** Esta restricción garantiza que los valores en una columna o combinación de columnas sean únicos en la tabla, lo que significa que no puede haber duplicados en esa columna o combinación de columnas.
- **Not Null Constraint (Restricción de no nulos):** Esta restricción asegura que una columna no pueda tener valores nulos, lo que significa que se debe proporcionar un valor válido para esa columna al insertar o actualizar datos.

- **Check Constraint (Restricción de comprobación):** Esta restricción permite especificar una condición que debe cumplirse para los valores en una columna. Solo se permiten los valores que cumplan con la condición especificada en la restricción.

Estas restricciones se definen durante la creación de una tabla o se agregan más tarde utilizando la sentencia ALTER TABLE. Su propósito es garantizar la consistencia de los datos y prevenir operaciones que violen las reglas establecidas.

Al aplicar estas restricciones en una tabla SQL, se asegura la integridad y calidad de los datos almacenados, evitando inconsistencias y asegurando que los datos cumplan con las reglas definidas para cada columna o combinación de columnas.

Para aplicar la integridad referencial, se pueden definir restricciones de clave externa entre tablas. Estas restricciones especifican las relaciones entre las tablas y definen las acciones que deben llevarse a cabo cuando se producen determinadas operaciones, como la inserción, la actualización o la eliminación.

Insertar, actualizar y eliminar datos con integridad referencial

Para insertar, actualizar y eliminar datos con integridad referencial en SQL hay algunos pasos que puedes seguir:

Al **insertar datos** en una tabla con una columna de clave externa, hay que asegurarse de que el valor referenciado existe en la columna de clave primaria de la tabla padre.

Ejemplo:

```
INSERT INTO ChildTable (child_id, parent_id,
data) VALUES (1, 100, 'Some data');
```

Al **actualizar datos** en una tabla con una columna de clave ajena, tenga cuidado de no modificar los valores de clave ajena a referencias inexistentes o no válidas.

Ejemplo:

```
UPDATE ChildTable SET
parent_id = 200 WHERE
child_id = 1;
```

Al **eliminar datos** de una tabla con una columna de clave externa, hay que tener en cuenta las restricciones de integridad referencial para mantener la coherencia de los datos.

Dependiendo del comportamiento deseado, se pueden utilizar acciones en cascada en las restricciones de clave externa.

Ejemplo de uso de CASCADE:

```
DELETE FROM ParentTable  
WHERE parent_id = 100;
```

Es importante tener en cuenta que tanto la sintaxis como el comportamiento específico para mantener la integridad referencial pueden variar según el sistema de base de datos que utilice. Es importante consultar la documentación de su sistema de base de datos específico para obtener detalles precisos.

Hay que asegurarse de que las referencias a claves externas sean válidas al insertar o actualizar datos y tiene en cuenta las acciones en cascada adecuadas al eliminar datos, puede mantener la integridad referencial en su base de datos SQL. Esto ayuda a preservar las relaciones entre tablas y garantiza operaciones de datos coherentes y fiables.

Transaccionalidad en las operaciones

¿Qué es una transacción y por qué son importantes?

La transaccionalidad en las operaciones en SQL se refiere a **la capacidad de agrupar una secuencia de operaciones de base de datos en una única transacción**, asegurando que son tratadas como **una única unidad de trabajo**. Este concepto es específico del lenguaje SQL y de los sistemas de bases de datos que lo soportan.

La transaccionalidad en las operaciones SQL se rige por las propiedades **ACID** (Atomicity, Consistency, Isolation y Durability):

- **Atomicidad:** Garantiza que una transacción sea tratada como una unidad indivisible. Todas las operaciones de una transacción se consignan en la

base de datos como un todo o no se aplica ninguna. Si alguna parte de la transacción falla, se revierte toda la transacción y la base de datos se mantiene sin cambios.

- **Consistencia:** La coherencia garantiza que una transacción lleve la base de datos de un estado válido a otro estado válido. La base de datos debe satisfacer todas las restricciones y reglas de integridad definidas antes y después de la transacción.
- **Aislamiento:** El aislamiento garantiza que las transacciones concurrentes no interfieran entre sí. Cada transacción se aísla de las demás hasta que se completa. Esto evita problemas como las lecturas sucias, las lecturas no repetibles y las lecturas fantasma.
- **Durabilidad:** La durabilidad garantiza que, una vez confirmada una transacción, sus cambios son permanentes y sobrevivirán a cualquier fallo posterior del sistema. Los cambios realizados en una transacción se almacenan de forma duradera y no se pierden.

Al adherirse a los principios de la transaccionalidad, SQL garantiza la coherencia y la integridad de los datos. Permite ejecutar como una sola unidad operaciones complejas en las que intervienen varias sentencias de la base de datos, lo que proporciona fiabilidad y capacidad de recuperación ante fallos o errores. Si una transacción falla, puede volver al estado anterior, lo que evita actualizaciones parciales o incoherentes de la base de datos.

¿Cómo confirmar y revertir las transacciones?

En SQL, suele haber dos tipos de transacciones:

Transacciones implícitas: Las transacciones implícitas son gestionadas automáticamente por el sistema de base de datos. En este modo, cada sentencia SQL se trata como una transacción independiente. El sistema de base de datos inicia automáticamente una transacción antes de ejecutar la sentencia y confirma o revierte la transacción una vez finalizada la sentencia. Si se produce un error, la transacción se revierte; en caso contrario, se consigna.

Transacciones explícitas: Las transacciones explícitas son gestionadas manualmente por el programador. En este modo, el programador inicia explícitamente una transacción, ejecuta varias sentencias SQL dentro de la transacción y confirma o revierte explícitamente la transacción en función del resultado deseado.

Dentro de las transacciones explícitas, existen otras opciones:

Para iniciar una transacción explícita, la sintaxis varía en función del sistema de base de datos. Por ejemplo:

- En SQL Server: BEGIN TRANSACTION;
- En Oracle: BEGIN;
- En MySQL: START TRANSACTION;

COMMIT: Guarda los cambios realizados dentro de la transacción, haciéndolos permanentes en la base de datos. La sintaxis es la misma en todos los sistemas de bases de datos: COMMIT

ROLLBACK: Deshace los cambios realizados en la transacción, devolviendo la base de datos a su estado anterior. La sintaxis es la misma en todos los sistemas de bases de datos: ROLLBACK

Al gestionar explícitamente las transacciones, se tiene un mayor control sobre los límites y los resultados de las operaciones de la base de datos.

Cierre

Durante esta lección, hemos aprendido sobre identificadores de bases de datos, secuencias e integridad referencial, y sobre los sistemas de bases de datos que pueden mantener la integridad de los datos y las relaciones entre las tablas.

Los identificadores proporcionan nombres únicos a los objetos, las secuencias generan identificadores únicos y las restricciones de integridad referencial garantizan la validez y coherencia de las relaciones entre tablas. Todos estos conceptos contribuyen a una gestión fiable y precisa de los datos en las bases de datos.

¡Muchas gracias!

Nos vemos en la próxima lección

